



Generative AI

Generated on: 2026-05-24 03:58:38 GMT+0000

SAP AI Core | For External SAP Customers

Public

Original content: https://help.sap.com/docs/AI_CORE/b9f48eb4a993445b863a55dd4d38f64d?locale=en-US&state=PRODUCTION&version=CLOUD

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit <https://help.sap.com/docs/disclaimer>.

Vertex AI

Context

You can access LLMs through the `foundation-models` and `orchestration` scenarios.

To use a model through the `orchestration` scenario, using the harmonized API, see [Orchestration](#).

For more information about available models, including conversion rates for tokens, rate limits and deprecation dates, see SAP Note [3437766](#) .

Accessing Models through Orchestration

Orchestration offers a harmonized API that allows you to use different models without changing the client code. You likely have an orchestration deployment running in your default resource group. If you want to access models through orchestration in a different resource group, you'll need to create an orchestration deployment for your chosen resource group. For more information, see [Create a Deployment for Orchestration](#).

Access to orchestration of generative AI models is provided under the global AI scenario `orchestration`, which is managed by SAP AI Core.

To access generative AI models using orchestration, you'll need to add following information to the `model` module of your orchestration workflow:

- The name of your chosen model
- The version name of your chosen version. If no model version is listed, it is not applicable.

For more information about supported models and associated costs, see SAP Note [3505347](#) .

For more information about orchestration workflows, see [Orchestration Workflow V2](#).

Accessing Models Through the `foundation-models` Scenario

You can access foundation models by creating a deployment for the model that you want to use. To do this, you'll need an auth token from your SAP AI Core instance. For more information, see [Get an Auth Token](#) and [Create a Deployment](#).

To use a specific version of a model, or to upgrade model versions manually, specify the model version your model deployment. To upgrade automatically, use model version: `latest`. For more information, see [Model Lifecycle](#). If no model version is listed, it is not applicable.

Vertex AI models are remote models.

The `executableId` is `gcp-vertexai`

After creating a deployment for your model, you consume the model using prompts. To access the model, you'll need your deployment ID, this can be set as an environment variable.

Ensure that you've set the following headers:

Header	Value
Authorization	Bearer \$AUTH_TOKEN
AI-Resource-Group	The resource group used in the activation steps
\$DEPLOYMENT_URL	The deployment URL for your generative AI model. For more information, see Create a Deployment. Alternatively, you can replace the \$DEPLOYMENT_URL placeholder in the curl command with your deployment URL.

Consumption

The following examples show how you can consume various generative AI models using curl. For more information about prompts, see the tutorial [Prompt LLMs in the Generative AI Hub in SAP AI Core & Launchpad](#).

→ Tip

If you use a Windows device, use Windows PowerShell, and replace curl with curl.exe.

For more information about supported parameters, see [Supported Parameters](#).

Gemini

For more information from the model provider, see [Vertex AI Gemini](#).

Gemini 2.0 Flash | Gemini 2.0 Flash Lite | Gemini 2.5 Flash | Gemini 2.5 Flash Lite | Gemini 2.5 Pro | Gemini 3.1 Flash Lite

Text Input

```
curl --request POST --location "$DEPLOYMENT_URL/models/<modelName>:generateContent" \ #example gem:
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
  "generation_config":{
    "maxOutputTokens":100
  },
  "contents": {
    "role": "user",
    "parts": {
      "text": "Give me a recipe for banana bread."
    }
  },
  "safety_settings": {
    "category": "HARM_CATEGORY_SEXUALLY_EXPLICIT",
    "threshold": "BLOCK_LOW_AND_ABOVE"
  }
}'
```

Image input

```
curl --request POST --location "$DEPLOYMENT_URL/models/<modelVersion>:generateContent" \ #example g
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
  "contents": {
```

```

    "role": "user",
    "parts": [
      {
        "fileData": {
          "mimeType": "image/png",
          "fileUri": "filepath/images/scones.jpg"
        }
      },
      {
        "text": "Describe this picture."
      }
    ]
  }
}'

```

Image Generation with Gemini 2.5 Flash

```

curl --request POST --location "$DEPLOYMENT_URL/models/gemini-2.5-flash-image:generateContent" \
--header "AI-Resource-Group: $AI_RESOURCE_GROUP" \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
  "contents": [
    {
      "role": "user",
      "parts": [
        {
          "text": "A cozy, cinematic coffee shop interior at golden hour, shallow depth of field, 3
        }
      ]
    }
  ]
}'

```

Gemini Embedding

```

curl --location '$DEPLOYMENT_URL/models/gemini-embedding:predict' \
--header 'AI-Resource-Group: <Resource Group Id>' \
--header 'Content-Type: application/json' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--data '{
  "instances": [
    {
      "task_type": "<Task Type>",
      "title": "Behaviour Analysis",
      "content": "What is quid pro quo?"
    }
  ]
}'

```

Task_type can take the following values:

- RETRIEVAL_QUERY
- RETRIEVAL_DOCUMENT
- SEMANTIC_SIMILARITY
- CLASSIFICATION
- CLUSTERING

Removing a Model

If you want to remove a model, delete its deployment. For more information, see [Delete a Single Deployment](#)

SAP-RPT-1

SAP - RPT - 1 is a relational pretrained transformer for use on relational and structured data. It's developed and maintained by SAP.

Relational Foundation Models (RFMs) are large-scale machine learning models designed to understand, process, and do predictions on tabular and relational data.

SAP - RPT - 1 is a table-native model aiming to achieve highest prediction quality and lowest error rates for predictions on tabular business data. It's pretrained, and doesn't need additional training or fine-tuning steps.

RPT-1 solves predictive tasks such as classification and regression out-of-the-box without requiring any training or fine-tuning via in-context learning. Due to its table-native architecture, prediction quality on enterprise data is typically very high, ahead of state-of-the-art narrow AI models and LLMs employed for such tasks.

You can get predictions directly in one API call from provided context examples.

Key Features

Key features include:

- **Pretrained Model:** Removes the need to collect training datasets, manage compute infrastructure, or wait weeks for model training cycles. Deploy immediately with production-ready accuracy.
- **In-Context Learning:** Accepts representative context examples during inference time to return instant predictions without the need for setup or customization for your specific use cases.
- **Fast Iterations and Adaptation:** Adapts to the context data provided in your requests, without the need for additional training steps or deployments.
- **Grounded to Business:** Produces predictions that are grounded in the provided context examples, resulting in a lower risk of prediction errors or hallucinations.

Tabular In-Context Learning

SAP - RPT - 1 produces predictions through tabular in-context learning.

Prompts are given in table form. They contain context examples and fields marked for prediction with a placeholder value.

❖ Example

The following prompt shows a table where cost centers are associated with ordered products. A placeholder value ([PREDICT]) has been used to request a prediction for the COSTCENTER of the couch. SAP - RPT - 1 returns proposals for the prediction request. Multiple requests can be included in one table prompt.

PRODUCT	PRICE	ORDERDATE	ID	COSTCENTER
Couch	999.99	28-11-2025	35	[PREDICT]
Office Chair	150.8	02-11-2025	44	Office Furniture
Server Rack	2200.00	01-11-2025	104	Data Infrastructure

SAP RPT Models

SAP - RPT - 1 has the following versions:

- `sap-rpt-1-small`, which is recommended when:
 - Prediction scenarios are of medium complexity.
 - Low latency and high-prediction throughput are the primary objective.
- `sap-rpt-1-large`, which is recommended when:
 - Prediction scenarios are complex.
 - Best prediction quality and lowest error rates are the primary objective.

Model Specifications

Value	<code>sap-rpt-1-small</code>	<code>sap-rpt-1-large</code>
Max. Context Length	2048 rows	65536 rows
Max. Columns	100 columns	256 columns
Number of Target Classes (Classification)	256*	1024*
Recommended Context Length	500–2000 rows ⁺	4000–8000 rows ⁺

Both model versions allow up to 128 simultaneous prediction rows and up to 10 simultaneous prediction columns in each prediction request. Supported task types are `classification` and `regression`.

* The number of target classes is a recommendation for best prediction quality, not a hard limit. Depending on the specific use case, models can produce high-quality predictions for larger numbers of target classes.

+ Generic recommendations on context length are best practice values based on internal tests, balancing prediction quality, model runtime, and costs. Optimal settings vary across use cases and data sets, so customers or implementation partners must conduct their own testing with production data to achieve the best results for their specific scenario.

Workflow

You can access the models through the `foundation-models` scenario and the `aicore-sap` executable ID.

To use a model through the `foundation-models` scenario, you must create a configuration and then a deployment. You need the `executableId`: `aicore-sap` and `modelName`: `sap-rpt-1-small` or `sap-rpt-1-large`. You will also need a model version, or you can use `latest` to use the latest version. For more information, see [Create a Deployment](#).

You consume the model through API calls. For more information, see [Example Payloads for Inferencing: sap-rpt-1](#).

Costs

For an estimate of projected costs, use the For an estimate of projected costs, use the SAP AI Core cost calculator, and select your preferred model. For more information, see [SAP AI Core Cost Calculator for Generative AI](#).

Example Payloads for Inferencing: sap-rpt-1

→ Tip

If you use a Windows device, use Windows PowerShell, and replace `curl` with `curl.exe`.

Ensure that you've set the following headers:

Header	Value
Authorization	Bearer \$AUTH_TOKEN
AI-Resource-Group	The resource group used in the activation steps
\$DEPLOYMENT_URL	The deployment URL for your generative AI model. For more information, see Create a Deployment. Alternatively, you can replace the \$DEPLOYMENT_URL placeholder in the curl command with your deployment URL.

The `executableId` is `aicore-sap`.

Best Practices

To achieve the best possible outputs, ensure that your requests comply with the following best practices:

- Use clean data with no or minimum missing values.
- Use descriptive column names (under 100 characters).
- Limit descriptions in text columns to 500 characters.
- Keep data types consistent within each column.
- Consider removing unnecessary columns from the prediction task.

i Note

The accuracy and fairness of the model's predictions depend on the quality and balance of the data provided. Any bias present in the input data can result in biased or skewed outputs. SAP isn't responsible or liable for any bias, inaccuracy, or unintended outcome arising from user-provided data or its characteristics.

Request Payloads

All JSON payloads must be UTF-8 encoded.

`sap-rpt-1-small` and `sap-rpt-1-large` accept tabular data requests in the following formats:

- Data as JSON organized by rows
- Data as JSON organized by columns
- Data in Apache Parquet file format

You can make requests using the following methods:

Request with Data by Rows

```
curl --request POST \  
  --url "$DEPLOYMENT_URL/predict" \  
  --data @data.json
```

```
--header 'AI-Resource-Group: default' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--header 'content-type: application/json' \
--data '{
  "prediction_config": {
    "target_columns": [
      {
        "name": "COSTCENTER",
        "prediction_placeholder": "[PREDICT]",
        "task_type": "classification",
        "top_k": 1
      }
    ]
  },
  "index_column": "ID",
  "rows": [
    {
      "PRODUCT": "Couch",
      "PRICE": 999.99,
      "ORDERDATE": "28-11-2025",
      "ID": "35",
      "COSTCENTER": "[PREDICT]"
    },
    {
      "PRODUCT": "Office Chair",
      "PRICE": 150.8,
      "ORDERDATE": "02-11-2025",
      "ID": "44",
      "COSTCENTER": "Office Furniture"
    },
    {
      "PRODUCT": "Server Rack",
      "PRICE": 2200.00,
      "ORDERDATE": "01-11-2025",
      "ID": "104",
      "COSTCENTER": "Data Infrastructure"
    }
  ],
  "data_schema": {
    "PRODUCT": {
      "dtype": "string"
    },
    "PRICE": {
      "dtype": "numeric"
    },
    "ORDERDATE": {
      "dtype": "date"
    },
    "ID": {
      "dtype": "string"
    },
    "COSTCENTER": {
      "dtype": "string"
    }
  }
}'
```

Request with Data by Columns

```
curl --request POST \
--url "$DEPLOYMENT_URL/predict" \
--header 'AI-Resource-Group: default' \
--header "Authorization: Bearer $AUTH_TOKEN" \
--header 'Content-Type: application/json' \
--data '{
  "prediction_config": {
    "target_columns": [
      {
        "name": "COSTCENTER",
        "prediction_placeholder": "[PREDICT]",
        "task_type": "classification",
        "top_k": 1
      }
    ]
  },
  "index_column": "ID",
  "rows": [
    {
      "PRODUCT": "Couch",
      "PRICE": 999.99,
      "ORDERDATE": "28-11-2025",
      "ID": "35",
      "COSTCENTER": "[PREDICT]"
    },
    {
      "PRODUCT": "Office Chair",
      "PRICE": 150.8,
      "ORDERDATE": "02-11-2025",
      "ID": "44",
      "COSTCENTER": "Office Furniture"
    },
    {
      "PRODUCT": "Server Rack",
      "PRICE": 2200.00,
      "ORDERDATE": "01-11-2025",
      "ID": "104",
      "COSTCENTER": "Data Infrastructure"
    }
  ],
  "data_schema": {
    "PRODUCT": {
      "dtype": "string"
    },
    "PRICE": {
      "dtype": "numeric"
    },
    "ORDERDATE": {
      "dtype": "date"
    },
    "ID": {
      "dtype": "string"
    },
    "COSTCENTER": {
      "dtype": "string"
    }
  }
}'
```



```

    }
  ]
},
"index_column": "ID",
"columns": {
  "PRODUCT": ["Couch", "Office Chair", "Server Rack"],
  "PRICE": [999.99, 150.8, 2200.00],
  "ORDERDATE": ["28-11-2025", "02-11-2025", "01-11-2025"],
  "ID": ["35", "44", "104"],
  "COSTCENTER": ["[PREDICT]", "Office Furniture", "Data Infrastructure"]
},
"data_schema": {
  "PRODUCT": {
    "dtype": "string"
  },
  "PRICE": {
    "dtype": "numeric"
  },
  "ORDERDATE": {
    "dtype": "date"
  },
  "ID": {
    "dtype": "string"
  },
  "COSTCENTER": {
    "dtype": "string"
  }
}
}'

```

Request with Parquet File

Parquet files can be uploaded using multipart form-data.

The request must include the following form fields:

- **file**: The parquet file to be uploaded (Content-Type: `application/vnd.apache.parquet`)
- **prediction_config**: A JSON string containing the prediction configuration
- **index_column**: The name of the column used to identify the row
- **parse_data_types**: A boolean that specifies whether to parse data types automatically (default: `true`)

The values for the `index_column` and `parse_data_types` fields are optional but recommended. If you don't specify values for these fields, the system uses default values.

The request must use `Content-Type: multipart/form-data` with an appropriate boundary string. The parquet file is read and processed similarly to row-wise and column-wise JSON inputs.

```

POST $DEPLOYMENT_URL/predict_parquet
--header "Content-Type: multipart/form-data; boundary=----WebKitFormBoundary7MA4YWxkTrZu0gW" \
--header "Authorization: Bearer $AUTH_TOKEN" \
--header "ai-resource-group: default"

-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="file"; filename="test_data.parquet"
Content-Type: application/vnd.apache.parquet

< ./test_data.parquet
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="prediction_config"

{"target_columns": [{"name": "salary", "prediction_placeholder": "[PREDICT]"}]}

-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="index_column"

```

```

name
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="parse_data_types"

true
-----WebKitFormBoundary7MA4YWxkTrZu0gW--

```

Parameters

Parameter Description for /predict (JSON input)

Name	Type	Description
prediction_config	object	The configuration object specifying which columns to predict
prediction_config.target_columns	array	An array of target-column configurations for prediction
prediction_config.target_columns[].name	string	The name of the column to predict
prediction_config.target_columns[].prediction_placeholder	string	The value used as placeholder in rows where prediction is needed
prediction_config.target_columns[].task_type	string	(Optional) The type of prediction task Possible values: classification or regression If omitted, a regression task is assumed if at least one value in the input data is formatted like a float, (i.e. "1.0"), otherwise classification is performed.
prediction_config.target_columns[].top_k	integer	(Optional) default is 1. Only for task type "classification". Controls how many labels to predict for each column in each row.
index_column	string	(Optional) The name of the column used to identify the row This column isn't used as an input feature for the model, and is returned in the response objects.

Name	Type	Description
rows	array	An array of objects representing table rows (both context and query rows) The keys of the objects are column names, for example <pre>[{"product": "hammer", "price": 42.99, "creation_date": "2025-01-30"}, ...]</pre>
columns	object	An object mapping from column name to array of column values. You can pass rows only or columns only, for example <pre>{"product": ["hammer", "nail", ...], "price": [42.99, 1.98, ...], "creation_date": ["2025-01-30", "2025-01-31", ...]}</pre>
data_schema	object	(Optional) A schema definition for data types of all columns, for example <pre>{"columnA": {"dtype": "string"}, "columnB": {"dtype": "numeric"}, ...}</pre>
data_schema.*.dtype	string	A data type specification. Possible values: string, numeric, or date
parse_data_types	Boolean	Relevant when data_schema isn't passed Whether or not to parse data types, such as interpreting strings as numbers or dates Default value: true

The `data_schema` parameter and `task_type` parameters are optional, but are recommended to obtain the best prediction results.

If `data_schema` is omitted, data types are inferred automatically.

If `task_type` is omitted, the model attempts to determine whether `classification` or `regression` is the intended task type.

Parameter description for `/predict_parquet` (Parquet file input as multipart/form-data)

Name	Type	Description
<code>prediction_config</code>	object	The configuration object specifying which columns to predict
<code>prediction_config.target_columns</code>	array	The array of target-column configurations for prediction
<code>prediction_config.target_columns[].name</code>	string	The name of the column to predict
<code>prediction_config.target_columns[].prediction_placeholder</code>	string	The value used as placeholder in rows where prediction is needed
<code>prediction_config.target_columns[].top_k</code>	integer	(Optional) default is 1. Only for task type "classification". Controls how many labels to predict for each column in each row.
<code>prediction_config.target_columns[].task_type</code>	string	(Optional) The type of prediction task Possible values: classification or regression If omitted, the model tries to automatically determine the task type that the user intends.
<code>index_column</code>	string	(Optional) The name of the column used to identify the row This column isn't used as an input feature for the model, and is returned in the response objects.
<code>file</code>	application/	A parquet file

Response Payload

The objects in `predictions` are sequence-aligned with the order of the query rows in the request JSON.

If the `index_column` parameter has been passed, the contents of this column are also included in the response.

To align the response to the source data, you can sequence-align the arrays, or compare the contents of the index column where included.

Name	Type	Description
id	string	The generated UUID4 for this request (useful for logging)
status	object	The status message object indicating success of the prediction request
status.code	integer	The numeric status code. For more information, see Table 1 .
status.message	string	The status message text
predictions[*].prediction	string or float	The predicted value for this column
predictions[*].confidence	float or null	Confidence value for this classification prediction (null for regression)
predictions[*].confidence_interval	array or null	Confidence interval for this regression prediction as array of two floats [lower_bound, upper_bound] or null for classification
metadata.num_rows	integer	The total number of input rows
metadata.num_columns	integer	The total number of input columns
metadata.num_predictions	integer	The number of table cells containing the specified placeholder values summed over all target columns
metadata.num_query_rows	integer	The number of query rows for which a prediction was made

i Note

Error responses can contain an additional array called `detail`, following pydantic format. For more information, see [Table 1](#).

Status Codes and Messages

A status code is returned to indicate success, a warning, or an error. Status codes can be any of the following:

Status Codes and Messages		
Code	Message	Description
0	OK	Everything worked as expected.
1	Warning: At least one target column had invalid values in some context rows (expected float-like values for regression). These rows were dropped. Double-check your data; prediction accuracy may be impacted.	Sub optimal data/accuracy warning still returns predictions.
2	Invalid input	The input was invalid: It could contain, for example.

Code	Message	Description
		- Malformed JSON - Missing properties - Incorrectly typed properties - Maximum restrictions exceeded
3	Internal server error	An unexpected error occurred.

Successful Prediction (200)

If the parameter `index_column` is specified in the request JSON, the following apply:

- Each prediction object in the array contains an index column. In the following examples, the index column is called **ID**.
- The column referenced as the `index_column` isn't input to the model and serves as index information in the response JSON.

Output Code

```
{
  "id": "c334f854-0d70-4c79-bd73-9ac581fd8cda",
  "status": {
    "code": 0,
    "message": "ok"
  },
  "predictions": [
    {
      "COSTCENTER": [
        {
          "prediction": "Office Furniture",
          "confidence": 0.96,
          "confidence_interval": null
        }
      ],
      "ID": "35"
    }
  ],
  "metadata": {
    "num_columns": 5,
    "num_rows": 2,
    "num_predictions": 1,
    "num_query_rows": 1
  }
}
```

Successful Prediction (200) with Warning

This example shows an output where the request to perform a regression contains either empty strings or null values in the regression target column.

```
{
  "id": "bf9018c4-f903-4035-a433-7058cb289447",
  "status": {
    "code": 1,
    "message": "Warning: At least one target column had invalid values in some context rows (e"
  },
  "predictions": [
    {
      "salary": [
        {
          "prediction": 52451.3984375,

```

```

        "confidence": null,
        "confidence_interval": [50000.0, 58500.0]
    }
}
],
"metadata": {
    "num_columns": 4,
    "num_rows": 2,
    "num_predictions": 1,
    "num_query_rows": 1
}
}

```

Bad Request or Unprocessable Content (400 or 422)

```

{
  "status": {
    "code": 2,
    "message": "Invalid input"
  },
  "detail": [
    {
      "loc": [],
      "msg": "Too many query rows provided. Maximum is 128.",
      "type": "value_error"
    }
  ]
}

```

```

{
  "status": {
    "code": 2,
    "message": "Invalid input"
  },
  "detail": [
    {
      "loc": [
        "prediction_config",
        "target_columns",
        0,
        "prediction_placeholder"
      ],
      "msg": "Field required",
      "type": "missing"
    }
  ]
}

```

Internal Server Error (500)

⌵ Output Code

```

"status": {
  "code": 3,
  "message": "Internal server error"
}

```

SAP-ABAP-1

SAP-ABAP-1 is a foundation model built by SAP that helps with coding tasks. It is fine-tuned on a large amount of SAP ABAP code.

Key Features

Key features include:

- **Pretrained Model:** The model is fine-tuned on the SAP S/4HANA ABAP and CDS code base. There's no need to collect, prepare, or manage training data. You also don't need to manage costly compute infrastructure for training.
- **Code Explanation:** The model helps you to understand code by turning code to text. For more information, see [Using SAP-ABAP-1 to Explain Code](#).
- **Integration:** You can integrate the model into development tools, IDEs, or agentic frameworks.
- **Access to SAP Internal Models:** You gain direct access to the model used internally for the SAP Joule for developers feature in ADT. The same model is also used for SAP Joule for Consultants and the Custom Code Migration scenario, which explains old code to users during the migration from SAP ERP to SAP S/4HANA.

Accessing SAP-ABAP-1

SAP - ABAP - 1 can only be accessed via the Orchestration service of the generative AI hub. Make sure you have met the following prerequisites before you start:

- You are onboarded to SAP AI Core and have an SAP AI Core service instance and service key. For more information, see [Initial Setup](#).
- You are using the extended service plan. For more information, see [Service Plans](#) and [Update a Service Plan](#).
- You have completed the client authorization for your preferred user interface. For more information, see [Use a Service Key](#).
- You have obtained your Orchestration service deployment URL. For more information, see [Get An Orchestration Deployment URL](#).

You can submit a test call to SAP - ABAP - 1 using the Orchestration service. To do so, use `sap-abap-1` as the `model_name` in the request payload.

Using SAP-ABAP-1 to Explain Code

After you have met the prerequisites, you can make inference calls. SAP - ABAP - 1 supports the code-to-text scenario. The input is an ABAP class or a code snippet from your ABAP code repository. The output is an explanation in English of this class or code snippet.

For more information about how to make inference calls, see [Example Payloads for Inferencing: SAP-ABAP-1](#).

Context Length

The maximum context length is 128,000 tokens. Considering the need to reserve some context length for prompt instructions and output, the maximum input code size is approximately 120,000 tokens. This corresponds to roughly 250,000 input code characters.

Input/Output Filtering

Input and output filtering is configured for the model to prevent malicious or harmful content being included in the input or output.

FAQ

1. What is the relationship between **SAP-ABAP-1** and the SAP Joule for developers feature in the ABAP Development Tools for Eclipse?

SAP-ABAP-1 is used internally for the SAP Joule for developers feature in the ABAP Development Tools for Eclipse.

2. Can **SAP-ABAP-1** also be used to generate code?

The current version of SAP-ABAP-1 is optimized for explaining ABAP code. It can also generate code, but this capability is experimental and not recommended for productive use. You're welcome to explore code generation, but please note that it's not the primary focus of the model.

3. Can the model be integrated into the ABAP Development Tools for Eclipse, VS Code, or other IDEs?

The model is natively integrated into the ABAP Development Tools for Eclipse (ADT) as part of SAP Joule for developers. Currently, there isn't an out-of-the-box integration for VS Code. However, it's technically possible to integrate with any IDE that has extension interfaces.

4. Is it possible to access the model weights for further fine-tuning?

No, currently SAP-ABAP-1 is only accessible through the Orchestration service of the generative AI hub.

5. Can I test **SAP-ABAP-1** for free?

Yes, you can test the model for 30 days free of charge by using the "Try now" feature. For more information, see [Generative AI Hub](#) .

Related Information

[Example Payloads for Inferencing: SAP-ABAP-1](#)

Example Payloads for Inferencing: SAP-ABAP-1

Inferencing on the SAP-ABAP-1 model through the generative AI hub Orchestration service can be done with a POST HTTP call to the `ORCHESTRATION_SERVICE` endpoint with an appropriate payload and API headers. For information about the headers, request payload, and expected response, see [Orchestration Workflow V2](#).

The Orchestration service offers multiple configuration options. From these options, `prompt_templating` and `model` configuration are mandatory.

→ Tip

If you use a Windows device, use Windows PowerShell, and replace `curl` with `curl.exe`.

Best Practices

- Code with syntax errors or missing dependencies produces poor results. Use the model on finished code instead.
- Including inline code documentation enhances outcomes.
- If you're interested in understanding a specific part of your class, provide only that section as input. Supplying very long classes in their entirety may lead to sub-optimal results.
- Avoid inputting commented-out code because it can confuse the model.
- Only input ABAP code – other programming languages aren't supported.

Request Payloads

The curl script below shows a typical sample call to explain an ABAP class.

```
curl --request POST \
  --url "$ORCHESTRATION_SERVICE_URL/v2/completion" \
  --header 'ai-resource-group: default' \
  --header "authorization: Bearer $AUTH_TOKEN" \
  --header 'Content-Type: application/json' \
  --data '{
    "config": {
      "modules": {
        "prompt_templating": {
          "prompt": {
            "template": [
              {
                "role": "user",
                "content": "Explain the ABAP class in the input code below.\n\nThe explanation should I
            ]
          },
          "model": {
            "name": "sap-abap-1",
            "version": "latest",
            "params": {
              "temperature": 0.1,
              "max_tokens": 2000
            }
          }
        }
      }
    }
  }'
```

Request Parameter Descriptions

All JSON payloads must be UTF-8 encoded.

Name	Type	Description
ORCHESTRATION_SERVICE_URL	string	Orchestration deployment URL (see Get An Orchestration Deployment URL)
AUTH_TOKEN	string	Bearer token generated using the SAP AI Core service key
config.modules.prompt_templating.prompt.template	string	For role:user, the field content holds the actual explanation prompt with the input code (see Prompting Templates).
config.modules.prompt_templating.model.name	string	Always sap-abap-1
config.modules.prompt_templating.model.name	string	latest
config.modules.prompt_templating.model.params.temperature	float	LLM temperature. Good explanations are returned with a value of 0.1, but use 0.0 if you want deterministic results.

Name	Type	Description
config.modules.prompt_templating.model.params.max_tokens	integer	Maximum number of output tokens. The model stops predicting when this number is reached. A token is the smallest processing unit of the LLM. On average these LLM tokens are between 2-3 characters.

For more information about input parameters, see [Orchestration v2](#) on the Business Accelerator Hub.

Response Payloads

The response payload, along with the `request_id`, contains two major blocks: `intermediate_results` and `final_result`. `intermediate_results` lists all the intermediate results from the configured Orchestration modules. `final_result` contains the final output after the configured modules have been applied. The final explanation can be found in the `final_result` block in the `content` field.

The response to the example shown above is as follows:

```
{
  "request_id": "a0dd9ce1-c9d2-986c-9c0b-2c93d47b8bc0",
  "intermediate_results": {
    "templating": [
      {
        "role": "user",
        "content": "Explain the ABAP class in the input code below.\n\nThe explanation should have t
      ]
    ],
    "input_filtering": {
      "message": "Input Filter passed successfully.",
      "data": {
        "azure_content_safety": {
          "hate": 0,
          "self_harm": 0,
          "sexual": 0,
          "violence": 0,
          "user_prompt_analysis": {
            "attack_detected": false
          }
        }
      }
    },
    "output_filtering": {
      "message": "Choice 0: Output Filter passed successfully.\n",
      "data": {
        "choices": [
          {
            "index": 0,
            "azure_content_safety": {
              "hate": 0,
              "self_harm": 0,
              "sexual": 0,
              "violence": 0
            }
          }
        ]
      }
    },
    "llm": {
      "id": "chatcmpl-f4dbe242d8c84a8280b3d9411e29b460",
      "object": "chat.completion",
      "created": 1765309213,

```

```

    "model": "sap-abap-1",
    "choices": [
      {
        "index": 0,
        "message": {
          "role": "assistant",
          "content": "The class `ZCL_ADD_TEST` is a simple utility class designed to perform add:",
        },
        "finish_reason": "stop"
      }
    ],
    "usage": {
      "completion_tokens": 172,
      "prompt_tokens": 422,
      "total_tokens": 594
    }
  },
  "final_result": {
    "id": "chatcmpl-f4dbe242d8c84a8280b3d9411e29b460",
    "object": "chat.completion",
    "created": 1765309213,
    "model": "sap-abap-1",
    "choices": [
      {
        "index": 0,
        "message": {
          "role": "assistant",
          "content": "The class `ZCL_ADD_TEST` is a simple utility class designed to perform addit:",
        },
        "finish_reason": "stop"
      }
    ],
    "usage": {
      "completion_tokens": 172,
      "prompt_tokens": 422,
      "total_tokens": 594
    }
  }
}

```

Response Parameter Descriptions

Name	Type	Description
request_id	string	Generated UUID4 for this request (useful for logging)
intermediate_results	object	Intermediate results from orchestration service
final_result.id	string	Generated UUID4 for this response
final_result.object	string	Prediction object type
final_result.created	string	Creation timestamp
final_result.model	string	Always sap-abap-1
final_result.choices	object	List of predictions (typically, there is only one prediction needed and therefore only one element in this list)
index	integer	Index of element in prediction list (choices list)
message.role	string	Response role

Name	Type	Description
message.content	string	Actual prediction/explanation
message.finish_reason	string	Applied stopping condition. stop: the internal stopping condition from the LLM was reached. length: The maximum token length specified in the request was reached.
final_result.usage.completion_tokens	integer	Number of prediction tokens (for billing)
final_result.usage.prompt_tokens	integer	Number of input tokens (for billing)
final_result.usage.total_tokens	integer	Number of input plus output tokens (for billing)

For more information about the input parameters, output parameters, and status codes, see [Orchestration v2](#) on the Business Accelerator Hub.

Related Information

[SAP-ABAP-1](#)

Prompting Templates

SAP - ABAP - 1 can be used for different code explanation scenarios, for example:

- Explain a complete ABAP class
- Explain an ABAP method
- Explain an ABAP code snippet

For each of these scenarios, you can use one of the following prompt templates and adapt it to your needs. You can also allow additional user input so that users can formulate specific questions about the input code.

To submit these prompt templates to SAP - ABAP - 1 via the Orchestration service, insert the prompt template into the user prompt section of your prompt (see [Example Payloads for Inferencing: SAP-ABAP-1](#)).

Explain a Complete ABAP Class

You can use the following prompt template to get an explanation of a complete ABAP class. If you only want the definition or implementation section to be explained, include this instruction in the prompt instructions.

```
Explain the ABAP class in the input code below.
The explanation should have the following five sections.
- Introduce and summarize the class at the beginning in one paragraph.
- Provide a summary of the variables, without going into too much detail and provide individual headers for each variable.
- If no interfaces are implemented in the class, skip this section. Provide a summary of the interfaces implemented.
The overall header should be named `### Interfaces`.
- Provide a summary of each method, without going into too much detail and provide individual headers for each method.
- Give a brief conclusion that focuses on the usage of the class. Do not provide an overall header for the conclusion.
Input code:
```

```
```abap
{ abap_code_snippet }```
```

## Explain an ABAP Method

You can use the following prompt template to get an explanation of an ABAP method. The prompt takes the code for the method as the first input. It also takes the complete class code as additional context to provide the model with more information.

It can be useful to truncate the class and keep only the parts that are relevant to understand the code snippet.

```
Explain the ABAP method in the input code below.
The explanation should have the following four sections.
- Introduce and summarize the code at the beginning in one paragraph.
- Explain the signature of the method.
- Give a step-by-step technical explanation of what each block of the code is doing and provide inc
- Give a brief conclusion that focuses on method usage, returning variables and/or exception handl:
Input code:
```abap
{ abap_code_snippet }```
Here is the truncated form of the class in which the code snippet is part of. Use this as additio
```abap
```

## Explain an ABAP Code Snippet

You can use the following prompt template to get an explanation of a code snippet. The prompt takes the code snippet as the first input. It also takes the complete class code as additional context to provide the model with more information.

It can be useful to truncate the class and keep only the parts that are relevant to understand the code snippet.

```
Explain the ABAP input code snippet below.
The explanation should have the following two sections.
- Introduce and summarize the code snippet at the beginning in one paragraph.
- Give a step-by-step technical explanation of what each block of the code is doing and provide inc
Input code:
```abap
{ abap_code_snippet }```
Here is the truncated form of the class in which the code snippet is part of. Use this as additio
```abap
{ abap_class_code }```
```

## System Prompt

For security reasons, the system prompt is predefined and managed by SAP. Custom system prompts are not supported.

If a system prompt is included in the request, the request fails and an error is returned.

In typical scenarios, providing a user prompt is sufficient. If additional instructions are required, include them at the beginning or end of the user prompt.

## Adding Additional Context on the Input Code

Sometimes, the input code snippet or class alone doesn't provide enough context for the model to generate a good explanation. If you have additional context information about the input code, you can add it to the user prompt. For example, you can add information about the business purpose of the code, the module where the code is used, any other code documentation, more information about important code dependencies of the code, or any other relevant information that helps to understand the code better.